SOFTWARE DEVELOPMENT

Complete Learning Guide

From problem decomposition to reliable production delivery

Built for self-paced learning. Each module includes explanations, worked examples, workplace scenarios, common mistakes and knowledge checks—not just definitions.

Assessment guidance: 70% required to pass. Students can retake the assessment; focus on understanding why a choice is correct.

How to use this guide

Read each module in order. Trace examples by hand, predict the output before reading the explanation, and answer the checkpoint questions without looking back. Use the practice quiz to identify topics that need revision.

Habit	Practice
Clarify first	Write inputs, outputs, constraints and expected behaviour.
Trace examples	Use a small input and record each step.
Check edge cases	Test empty, boundary, duplicate and invalid inputs where allowed.
Prefer small changes	Make focused commits and refactor behind tests.
Validate releases	Test, monitor and prepare a rollback plan.

Contents

Module	Topic
01	Problem Solving Fundamentals
02	Data Structures
03	Algorithms & Complexity
04	Clean Code & Refactoring
05	Version Control in Practice
06	Testing & Code Review
07	System Design Basics
08	APIs & Integration
09	Delivery & DevOps Basics
Final	Practice quiz, explanations and revision sheet

Module 1: Problem Solving Fundamentals

A repeatable method instead of guessing.

What you should be able to do

• Restate a problem using inputs, outputs, constraints and examples.

• Break large problems into smaller tasks and test them independently.

• Use brute force as a baseline and reason about edge cases and invariants.

1. Understand before coding

Read the statement carefully. Identify the input format, expected output, constraints, and what counts as a valid result. Rewrite the problem in your own words. Ask clarifying questions about duplicates, empty input, negative values, ordering, and whether a solution must be optimal.

2. Problem decomposition

Split the task into smaller pieces with clear responsibilities. For a word-frequency task: read the words, normalise case if required, count occurrences, and display the result. Test each piece separately. Decomposition reduces cognitive load and makes bugs easier to locate.

3. Work an example by hand

Trace a small example with a table showing the current values, decisions, and output. Try an ordinary case and a boundary case. Manual tracing often exposes a mistaken assumption before code is written.

4. Brute force first

Brute force checks possibilities directly. It may be slow, but it is often simple to verify and makes a good baseline. Once it is correct, look for repeated work, unnecessary nested loops, sorting opportunities, or a data structure that makes a required operation faster.

5. Edge cases

Check empty input, one item, all equal values, duplicates, minimum/maximum constraints, already sorted or reverse-sorted data, negative values, and missing values where allowed. The relevant edge cases depend on the statement.

6. Invariants and correctness

An invariant is a condition that remains true at a particular point in an algorithm. In a running-sum loop, for example, after processing index i, the running total should equal the sum of all processed elements. State the invariant, initialise it correctly, and ensure each iteration preserves it.

Worked example / workplace scenario

Example: Find whether a list contains a duplicate. Brute force compares every pair: $O(n^2)$ time and O(1) extra space. A set-based approach checks whether each value has appeared before: expected O(n) time and O(n) extra space. First clarify whether values can be strings, nulls or repeated many times.

Common mistakes to avoid

• Coding before understanding constraints.

• Testing only the example given in the prompt.

• Optimising before establishing correctness.

• Treating an invariant as a comment rather than a property that can be checked.

Check your understanding

Q1. What four things should you identify before coding?

Q2. Why is brute force useful even if it is slow?

Q3. Name two useful edge cases for a list algorithm.

Suggested answers

A1. Inputs, outputs, constraints and expected behaviour.

A2. It gives a straightforward correctness baseline for comparison.

A3. Examples: empty list, one element, duplicates, all equal, negative values, or maximum size.

Remember: Reliable software comes from clear requirements, suitable structures, small verified changes and deliberate operational safeguards.

Module 2: Data Structures

Picking the structure that makes the problem easier.

What you should be able to do

• Compare arrays/lists, stacks, queues, maps and sets.

• Choose a structure based on required operations.

• Consider memory use and the trade-off between lookup and update costs.

1. Arrays and lists

Arrays store indexed elements and provide fast access by index, typically O(1). Inserting into the middle may require shifting elements, often O(n). Dynamic arrays can grow, although resizing occasionally costs extra work. Linked lists make insertion around a known node efficient, but accessing the kth element usually requires traversal.

2. Stacks

A stack follows last-in, first-out (LIFO). Push adds to the top and pop removes the top. Uses include undo history, function calls, parsing brackets and depth-first search. For balanced brackets, push opening symbols and pop when matching closing symbols appear.

3. Queues

A queue follows first-in, first-out (FIFO). Enqueue adds to the back; dequeue removes from the front. Uses include task scheduling, breadth-first search and handling requests in arrival order. A circular buffer or deque avoids repeatedly shifting an array.

4. Hash maps and sets

A hash map stores key-value pairs; a set stores distinct values. Lookup and insertion are expected O(1) on average, but can degrade under collisions or poor implementation. Use a map for frequency counts or ID-to-record lookup, and a set for membership or uniqueness checks.

5. Choosing by operation

Ask what the program does most: index-based access, membership tests, insertion/removal at ends, priority selection, or key-based retrieval. Choose for the dominant operation and verify the language library's guarantees.

6. Memory and trade-offs

A hash structure uses extra memory for buckets and metadata. Arrays may be compact and cache-friendly. A theoretically faster structure is not automatically best if the data is tiny, memory is constrained, or simpler code is safer.

Worked example / workplace scenario

Example: Count words. For each word, increment `count[word]`. A hash map is natural because the key is the word and the value is its frequency. To determine whether a value has appeared, a set is sufficient; storing a count would be unnecessary.

Common mistakes to avoid

• Using a stack when items must be processed in arrival order.

• Assuming hash operations are guaranteed O(1) in every case.

• Choosing a complex structure without considering actual operations.

Check your understanding

Q1. Which structure suits undo history?

Q2. Which structure suits breadth-first search?

Q3. Which structure suits checking whether an ID has appeared?

Suggested answers

A1. Stack.

A2. Queue.

A3. A set, or a map if associated information is also needed.

Remember: Reliable software comes from clear requirements, suitable structures, small verified changes and deliberate operational safeguards.

ROYSTIAN FOUNDATION

Module 3: Algorithms & Complexity

Knowing how a solution behaves as data grows.

What you should be able to do

• Express time and auxiliary space using Big-O.

• Apply binary search only when its preconditions hold.

• Recognise recursion, base cases and stack costs.

1. Big-O intuition

Big-O describes how cost grows as input size n grows, ignoring constant factors and lower-order terms. O(1) is constant, O(log n) grows slowly, O(n) is linear, O(n log n) is common for efficient comparison sorting, O(n^2) is quadratic, and O(2■) is exponential.

2. Count operations

A loop through n elements is usually O(n). Two independent loops are O(n+n)=O(n). A nested loop over n elements can be O(n^2). Repeatedly halving a search range gives O(log n). Analyse the dominant growth rather than timing one small test.

3. Binary search

Binary search compares the target with the middle element of a sorted range, then discards half the remaining range. Its time complexity is O(log n). It requires sorted data and correct boundary handling; using it on unsorted data is incorrect.

4. Recursion

A recursive function calls itself on a smaller problem. Every recursive solution needs a base case and progress toward that base case. Without them, recursion may never terminate or may overflow the call stack.

5. Space complexity

Time complexity estimates operation growth; space complexity estimates memory growth. Distinguish total memory from auxiliary memory if the question asks for extra space. A recursive function may use O(n) call-stack space even if it creates no explicit array.

6. Sorting and searching

Sorting can cost O(n log n) for common efficient comparison sorts and may enable binary search. If only one search is needed, sorting first may not be worthwhile. Choose the method based on data size, ordering and number of queries.

Worked example / workplace scenario

Trace binary search on sorted array [2, 5, 8, 12, 16, 23, 38] for target 16. Middle value 12 is too small, so search the right half. The next middle is 23, too large, so search left within that half. The remaining middle is 16: found. The range shrinks by roughly half each step.

Common mistakes to avoid

• Using binary search on unsorted input.

• Forgetting to move the low/high boundary after a comparison.

• Writing recursion without a base case.

• Confusing O(2n) with O(n^2): O(2n) simplifies to O(n).

Check your understanding

Q1. What is the time complexity of scanning a list once?

Q2. What condition does binary search require?

Q3. What two parts must every safe recursive solution have?

Suggested answers

A1. O(n).

A2. The searched range must be sorted and boundaries must be managed correctly.

A3. A base case and progress toward it.

Remember: Reliable software comes from clear requirements, suitable structures, small verified changes and deliberate operational safeguards.

Module 4: Clean Code & Refactoring

Writing for the next person who reads it.

What you should be able to do

• Use meaningful names and small, single-purpose functions.

• Recognise code smells and technical debt.

• Refactor safely while preserving behaviour.

1. Readable names

Names should communicate purpose and units. `elapsed_seconds` is clearer than `x`; `is_payment_valid` is clearer than `check`. Avoid misleading abbreviations and names that promise more than the variable contains.

2. Small functions and single responsibility

A function should have one clear job and a coherent level of abstraction. A function that validates an order, writes to a database, sends an email and formats a report has several reasons to change. Split responsibilities while keeping interfaces understandable.

3. Refactoring

Refactoring changes internal structure without changing externally observable behaviour. Examples include extracting a function, renaming a variable, removing duplication or simplifying a conditional. Make small changes and run tests after each meaningful step.

4. Code smells

Long functions, duplicated logic, deeply nested conditions, mysterious numbers, excessive parameters and unrelated responsibilities are warning signs. A smell is not automatic proof of a bug; use context and tests to decide what to improve.

5. Technical debt

A shortcut can reduce delivery time today but create future costs in debugging, change risk and maintenance. Make trade-offs explicit: document temporary workarounds, add a follow-up task and prioritise debt that repeatedly slows delivery or creates risk.

6. Comments and documentation

Prefer code that explains itself through names and structure. Comments are valuable for explaining why a non-obvious decision exists, business rules, constraints or trade-offs. Comments that merely repeat a line can become stale.

Worked example / workplace scenario

Example: Instead of a function named `doStuff(x)` that checks a user's age and returns a message, use `is_eligible_for_discount(age)` to return a boolean. Keep message formatting separate if multiple parts of the application need the eligibility rule.

Common mistakes to avoid

• Refactoring and changing behaviour at the same time.

• Creating tiny functions with unclear names or needless indirection.

• Removing a comment that explains a critical business constraint without preserving the reason elsewhere.

Check your understanding

Q1. Does refactoring intentionally change behaviour?

Q2. What does single responsibility mean?

Q3. Give one example of a code smell.

Suggested answers

A1. No; behaviour should remain the same.

A2. A unit should have one clear responsibility or reason to change.

A3. Duplicated logic, a long function, deep nesting or unexplained constants

Remember: Reliable software comes from clear requirements, suitable structures, small verified changes and deliberate operational safeguards.

Module 5: Version Control in Practice

Collaborating without stepping on each other.

What you should be able to do

• Use branches and atomic commits consistently.

• Understand rebase, tags and revert.

• Recover from common mistakes without losing shared work.

1. Git's mental model

A repository stores project history. A commit is a snapshot with metadata and a parent. A branch is a movable name pointing to a commit. The working tree contains current files; the staging area selects changes for the next commit.

2. Atomic commits

An atomic commit contains one coherent change. It should have a clear message such as `Validate email before account creation`. Small commits are easier to review, bisect and revert than one commit containing unrelated changes.

3. Branching and merging

Create a feature branch for a task, commit focused changes, then open a pull request for review. Resolve conflicts carefully and run tests after integration. Teams should agree on naming, review requirements and how the main branch is protected.

4. Rebase

Rebase reapplies commits onto a new base, often producing a linear history. It can be useful for cleaning up a local feature branch. Because it rewrites commit identities, avoid rebasing shared public history unless the team explicitly coordinates it.

5. Tags and releases

A tag marks a specific commit, commonly for a release such as v1.2.0. A release process can use tags to identify the source code associated with a deployed version.

6. Revert versus reset

`git revert` creates a new commit that undoes an earlier commit and is generally safer for shared history. `git reset` moves a branch pointer and may alter local history; modes can also affect the staging area or working tree. Understand the command before using it.

Worked example / workplace scenario

Example workflow: `git switch -c feature/login-validation`, edit files, `git add` only related changes, `git commit -m "Validate login input"`, push the branch, and open a pull request. If a bad commit is already shared, discuss and use `git revert ` rather than rewriting shared history.

Common mistakes to avoid

• Committing unrelated features together.

• Rebasing a branch other people already depend on without coordination.

• Using reset or force-push without understanding the effect.

• Writing messages such as `changes` or `fixed stuff`.

Check your understanding

Q1. What makes a commit atomic?

Q2. What is a tag commonly used for?

Q3. Why is revert often safer for shared history than reset?

Suggested answers

A1. It contains one coherent change that can be reviewed and undone independently.

A2. Marking a particular commit, often a release.

A3. It adds an undo commit without rewriting the shared commit history.

Remember: Reliable software comes from clear requirements, suitable structures, small verified changes and deliberate operational safeguards.

ROYSTIAN FOUNDATION

Module 6: Testing & Code Review

Habits that catch problems before production.

What you should be able to do

• Choose appropriate unit, integration and end-to-end tests.

• Give constructive code review feedback.

• Use regression tests and understand test-driven development.

1. Test pyramid

Unit tests are numerous, fast tests of small units. Integration tests check interactions between components. End-to-end tests exercise a complete user workflow and are usually slower and more fragile. A balanced suite uses each level for what it can verify best.

2. Test cases and assertions

A test arranges inputs, acts on the system and asserts expected behaviour. Include normal cases, boundaries, invalid inputs and failure paths. A test that runs without checking an outcome provides limited confidence.

3. Regression suite

Regression tests verify that previously working behaviour still works after changes. When a bug is fixed, add a test that would have failed before the fix when practical. This helps prevent the same defect from returning.

4. Test-driven development

TDD commonly follows red, green, refactor: write a failing test for desired behaviour, implement the smallest change to pass, then improve the design while tests remain green.

5. Code review

Review for correctness, edge cases, readability, security, performance and tests. Ask questions respectfully and explain impact. Separate blocking correctness issues from preferences, and respond to feedback with evidence rather than defensiveness.

6. Flaky tests and coverage

A flaky test passes and fails inconsistently, often because of timing, shared state or external dependencies. Coverage measures which code was exercised, not whether tests are meaningful. High coverage alone does not guarantee correctness.

Worked example / workplace scenario

Example: For a withdrawal function, test a valid withdrawal, an amount larger than the balance, zero, a negative amount and a missing account. Assert both returned results and important state changes, such as whether the balance changed.

Common mistakes to avoid

• Testing only the happy path.

• Treating code coverage as proof of quality.

• Writing tests that depend on execution order or real external services unnecessarily.

• Reviewing style while missing a correctness or security defect.

Check your understanding

Q1. Which test level checks interaction between services?

Q2. What does red-green-refactor mean?

Q3. Why add a regression test for a bug?

Suggested answers

A1. Integration testing.

A2. Write a failing test, make it pass, then refactor while keeping tests green.

A3. To detect the same bug if it returns.

Remember: Reliable software comes from clear requirements, suitable structures, small verified changes and deliberate operational safeguards.

Module 7: System Design Basics

Designing systems that can grow and recover.

What you should be able to do

• Separate service responsibilities and reason about scaling.

• Explain statelessness, caching and message queues.

• Identify single points of failure and mitigation options.

1. Layers and responsibilities

A system often separates client/UI, API, business logic, data access and storage. Clear boundaries make components easier to test and change. Split services when there is a reason—such as independent scaling or ownership—not merely to make the architecture look advanced.

2. Stateless services

A stateless application instance does not rely on local memory to remember a user's session between requests. Session state can live in a shared store or signed token, allowing a load balancer to send requests to different instances and making horizontal scaling easier.

3. Caching

A cache stores a result so repeated requests can be served faster and reduce load. Decide what can be cached, how long it stays valid, and how updates invalidate it. Stale data and cache stampedes are important risks.

4. Message queues

A queue buffers work between producers and consumers. For example, an order API can accept a request and enqueue email or report-generation work for asynchronous processing. Plan for retries, duplicate messages, dead-letter handling and eventual consistency.

5. Single point of failure

A component is a single point of failure if its failure alone can bring down the whole service. Examples include one database server with no failover or a single network gateway. Redundancy, health checks, backups, failover and tested recovery plans can reduce risk.

6. Capacity and trade-offs

At ten times the load, ask what saturates first: CPU, memory, database connections, storage, network or an external API. Measure before scaling. Replication and caching improve availability or speed but add cost and operational complexity.

Worked example / workplace scenario

Scenario: A ticket-booking site receives a sudden burst of requests. Stateless API instances can scale horizontally; caching popular read-only event details reduces repeated database queries; a queue can process confirmation emails asynchronously. Inventory updates still need strong coordination to avoid selling more tickets than exist.

Common mistakes to avoid

• Caching rapidly changing data without a freshness strategy.

• Assuming a queue guarantees exactly-once processing.

• Adding microservices before understanding the bottleneck.

• Relying on a backup that has never been restored in a test.

Check your understanding

Q1. Why does statelessness help horizontal scaling?

Q2. What is a queue useful for?

Q3. Give one way to reduce a single point of failure.

Suggested answers

A1. Any healthy instance can handle the next request without depending on local session state.

A2. Buffering and processing work asynchronously, decoupling producers and consumers.

A3. Redundant instances, failover, replicas, or a tested recovery path.

Remember: Reliable software comes from clear requirements, suitable structures, small verified changes and deliberate operational safeguards.

ROYSTIAN FOUNDATION

Module 8: APIs & Integration

Designing boundaries other teams can depend on.

What you should be able to do

• Define clear API contracts and version changes.

• Handle errors, retries, rate limits and timeouts predictably.

• Preserve backward compatibility for existing clients.

1. API contract

An API contract describes endpoints, methods, inputs, outputs, validation, authentication, errors and expected behaviour. Document required and optional fields, formats, units and examples. Consumers should not need to inspect the implementation to use the API correctly.

2. HTTP methods and status codes

GET retrieves a resource; POST commonly creates or triggers processing; PUT replaces a resource; PATCH partially updates it; DELETE removes it. Use status codes consistently: 200 success, 201 created, 400 invalid request, 401 unauthenticated, 403 forbidden, 404 not found, 429 rate limited, and 5xx server-side failures.

3. Versioning and compatibility

Adding an optional response field is often compatible, while renaming or removing a field can break clients. Versioning can use a URL path or another documented mechanism. Provide migration guidance and a deprecation period for breaking changes.

4. Rate limiting

Rate limits restrict request volume to protect capacity and ensure fair use. A client that receives 429 should respect retry guidance such as Retry-After when provided. Limits should be documented and applied consistently.

5. Timeouts and retries

Set timeouts so requests do not wait forever. Retry transient failures carefully with exponential backoff and jitter. Retrying a non-idempotent operation can create duplicates; use idempotency keys or safe operation design when appropriate.

6. Useful errors and observability

Return actionable, non-sensitive error details and a correlation/request ID. Do not expose secrets or stack traces to clients. Log enough context to diagnose failures while respecting privacy.

Worked example / workplace scenario

Example: `POST /v1/orders` accepts customer and item details, validates stock, and returns an order ID. If the client times out and retries, an idempotency key can prevent the same purchase from being created twice.

Common mistakes to avoid

• Changing a response field without checking consumers.

• Retrying every failure indefinitely.

• Returning sensitive internal details in error messages.

• Using status codes inconsistently.

Check your understanding

Q1. What is an API contract?

Q2. Why can retries create duplicate orders?

Q3. What should a client generally do after a 429 response?

Suggested answers

A1. The documented inputs, outputs and behaviour consumers rely on.

A2. The original request may have succeeded even if the response was lost; a retry can repeat the operation.

A3. Respect the rate limit and any Retry-After guidance, then retry safely.

Remember: Reliable software comes from clear requirements, suitable structures, small verified changes and deliberate operational safeguards.

Module 9: Delivery & DevOps Basics

Getting changes to users reliably.

What you should be able to do

• Describe the stages of a CI/CD pipeline.

• Use staging environments and feature flags appropriately.

• Plan safe releases, monitoring and rollback.

1. Continuous integration

CI automatically builds and tests changes as they are pushed or merged. A typical pipeline checks formatting, runs unit and integration tests, performs security or dependency checks, and builds an artifact. Fast feedback makes problems cheaper to fix.

2. Continuous delivery and deployment

Continuous delivery keeps software ready for release, often with a human approval step. Continuous deployment automatically releases changes that pass the pipeline. Both aim to make releases repeatable and reduce manual error.

3. Environments

Development is for active work; testing/QA validates behaviour; staging approximates production; production serves real users. Keep configuration and secrets managed appropriately, and avoid copying sensitive production data into lower environments without safeguards.

4. Feature flags

A feature flag enables or disables functionality without deploying new code. Flags support gradual rollout and quick disablement, but stale flags create complexity and should be reviewed and removed.

5. Blue-green deployment

Two production-like environments are maintained: one serves users while the other receives the new release. Traffic switches after validation. Rollback can switch traffic back, but database changes and external side effects must still be considered.

6. Monitoring and rollback

Monitor errors, latency, saturation and business metrics after release. Define rollback triggers in advance. A deployment is not fully safe just because it succeeded technically; user impact and data integrity matter too.

Worked example / workplace scenario

Example pipeline: developer pushes a branch → CI runs tests and checks → artifact is built → staging deployment → smoke tests and approval → gradual production rollout → monitor metrics. If error rate rises above the agreed threshold, stop rollout or roll back.

Common mistakes to avoid

• Assuming a green build guarantees the release is safe.

• Using production secrets in source code.

• Leaving feature flags permanently in place.

• Rolling back application code without considering database compatibility.

Check your understanding

Q1. What is CI designed to catch early?

Q2. How does a feature flag differ from a deployment?

Q3. What should be monitored after a release?

Suggested answers

A1. Build, test and integration problems before they reach production.

A2. It can enable or disable functionality at runtime without a new deployment.

A3. Errors, latency, resource saturation, and relevant business outcomes.

Remember: Reliable software comes from clear requirements, suitable structures, small verified changes and deliberate operational safeguards.

ROYSTIAN FOUNDATION

Final Assessment-Style Practice Quiz

Choose one best answer for each question. This practice quiz is not the platform's official assessment. A score of 14/20 equals 70%.

1. What should you clarify before coding?

A. Only the programming language

B. Inputs, outputs, constraints and expected behaviour

C. Variable names

D. UI colours

2. Why start with a brute-force solution?

A. It is always fastest

B. It provides a straightforward correctness baseline

C. It avoids testing

D. It uses no memory

3. Which structure is LIFO?

A. Queue

B. Stack

C. Set

D. Graph

4. Which structure is natural for membership checks?

A. Set

B. Queue

C. Stack only

D. Matrix only

5. A single scan over n items is usually:

A. O(1)

B. O(log n)

C. O(n)

D. $O(n^2)$

6. Binary search requires:

A. Unsorted input

B. Sorted search range

C. No boundary checks

D. A queue

7. A recursive solution needs:

A. Only a loop

B. A base case and progress toward it

C. A database

D. A global variable

8. Refactoring should:

A. Always add features

B. Change internal structure without changing behaviour

C. Remove all tests

D. Rewrite history

9. An atomic commit contains:

A. Every task in the sprint

B. One coherent change

C. Only documentation

D. No message

10. Which Git command creates a new commit that undoes an earlier commit?

A. git revert

B. git tag

C. git status

D. git log

11. What is the main role of a regression suite?

A. Measure colours

B. Check old behaviour still works after changes

C. Replace code review

D. Deploy automatically

12. Which is usually the largest layer of the test pyramid?

A. End-to-end tests

B. Unit tests

C. Manual production tests

D. Load balancers

13. A cache mainly helps by:

A. Guaranteeing fresh data forever

B. Serving repeated results faster and reducing repeated work

C. Replacing all databases

D. Removing errors

14. A queue is useful for:

A. Decoupling producers and consumers

B. Sorting all data automatically

C. Guaranteeing exactly-once delivery

D. Removing every failure

15. A single point of failure is:

A. Any slow function

B. A component whose failure alone can bring down the system

C. A unit test

D. A code comment

16. Why use an idempotency key for a payment request?

A. To make passwords shorter

B. To prevent retries from creating duplicate operations

C. To increase image size

D. To skip authentication

17. HTTP 429 generally indicates:

A. Created

B. Not found

C. Rate limited

D. Internal server error

18. A staging environment is used to:

A. Replace all tests

B. Validate a release in a production-like setup

C. Store passwords in code

D. Avoid monitoring

19. A feature flag lets a team:

A. Change functionality at runtime without a new deployment

B. Avoid all testing

C. Guarantee no bugs

D. Rewrite Git history

20. After a release, a team should monitor:

A. Only commit message length

B. Errors, latency, resource use and business impact

C. Only code formatting

D. Nothing if build passed

Answer Key and Explanations

Q	Answer	Explanation
1	B	These define the actual problem and its boundaries.
2	B	A simple baseline makes later optimisation easier to verify.
3	B	A stack removes the most recently added item first.
4	A	A set represents distinct values and supports efficient expected membership checks.
5	C	Work grows linearly with the number of items.
6	B	It discards half the range based on ordering.
7	B	These prevent non-termination and define the stopping condition.
8	B	Preserving external behaviour is the key constraint.
9	B	A focused commit is easier to review and revert.
10	A	Revert preserves shared history by adding an undo commit.
11	B	Regression tests detect previously fixed failures returning.
12	B	Unit tests are typically numerous and fast.
13	B	Cache freshness and invalidation still need careful design.
14	A	Queues buffer work and allow asynchronous processing.
15	B	Redundancy and recovery mechanisms can mitigate it.
16	B	The same logical request can be safely retried.
17	C	The client has exceeded a request limit.
18	B	It provides a pre-production validation step.
19	A	Flags control whether code paths are enabled.
20	B	A successful build does not guarantee a healthy production release.

Quick Revision Sheet

Topic	Key idea
Problem solving	Clarify requirements; decompose; trace examples; test edge cases; optimise after correctness.
Data structures	Array: indexed access; stack: LIFO; queue: FIFO; map: key-value; set: unique membership.
Complexity	O(1) constant; O(log n) halving; O(n) scan; O(n log n) common efficient sorting; $O(n^2)$ nested work.
Binary search	Requires sorted input and correct bounds.
Clean code	Meaningful names, focused functions, code smells as warning signs, refactor behind tests.
Git	Atomic commits, branches, tags for releases, revert for shared-history undo.
Testing	Unit tests are fast and numerous; integration tests check interactions; end-to-end tests cover workflows.
System design	Stateless services scale more easily; caches speed reads; queues decouple work; redundancy mitigates failure.
APIs	Clear contract, consistent status codes, safe retries, rate limits and compatibility.
DevOps	CI checks changes; staging validates; flags control rollout; monitor and prepare rollback.

Before you submit or release

- Are requirements and edge cases clear?
- Have I tested normal, boundary and failure cases?
- Can a reviewer understand each change and commit?
- Did I check complexity and memory for expected data size?
- Are API errors, retries and rate limits predictable?
- Are monitoring, rollback and data integrity covered?

Final principle: Good software is not only code that works once. It is code that can be understood, tested, changed, reviewed and operated safely.